

claude-windows / 2ba49afa-99f4-41f2-86a2-27a289d432f3

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\2ba49afa-99f4-41f2-86a2-27a289d432f3\subagents\workflows\wf_a48e6aa3-748\agent-abbc1d5dfbeda6f54.jsonl`  
- SHA-256: `ad3d588be8a962c1d8dbbace3c88b0cd69842b494584b19826e08949e10bbdf3`  
- Source modified: `2026-06-21T16:40:03+00:00`  
- Imported at: `2026-07-05T16:48:28+00:00`  
- project: `wf_a48e6aa3-748`  
- session_id: `2ba49afa-99f4-41f2-86a2-27a289d432f3`
```

Transcript

1. user (2026-06-21T16:31:36.790Z)

Review T4 for CORRECTNESS + edge cases. T4 (North Star keystone): in-UI local file UPLOAD wired to the engine's chunked-upload endpoints.

Files under C:/Users/avidu/Projects/khelsutra/web/src:

- api/client.ts: uploadInit/uploadPart(raw octet-stream PUT)/uploadComplete/uploadAbort + uploadFile(file,{cid,signal,onProgress}) orchestrator (init -> slice into max_part_mb parts -> sequential PUT 0,1,2 -> complete; best-effort DELETE on failure/abort).
- api/types.ts: UploadInit/Part/CompleteResponse.
- hooks/useUpload.ts: start(file) -> uploading{pct} -> onComplete(serverPath) | error(classifyError); rejects 0-byte; AbortController cancel/unmount.
- components/IngestPanel.tsx: file picker (primary) + the existing gs:// URI form (secondary); on upload complete -> useIngestJob.submit(path) (the SAME process->poll->onLoaded back-half); upload % progressbar; A8 chips reused.
- catalogs en/kn/hi: +ingest.chooseFile/uploadAria/fileHint/orLink/uploading.

Engine contract at C:/Users/avidu/Projects/badminton-highlight-indexer/backend/api/uploads.py. 150 web tests green; tsc + build clean.

Read C:/Users/avidu/Projects/khelsutra/web/src/api/client.ts (uploadFile + helpers) and C:/Users/avidu/Projects/khelsutra/web/src/hooks/useUpload.ts. Check rigorously:

1. CHUNKING math: file.slice(i*partBytes, min((i+1)*partBytes, size)) for i in 0..ceil(size/partBytes)-1 ? any off-by-one? Does the last part cover the remainder exactly? What if size is an exact multiple of partBytes (no trailing empty part)? What if max_part_mb is 0 or missing (Math.max(1,...) guard)?
2. SEQUENTIAL order: parts must be strictly 0,1,2 (the engine 409s on out-of-order). The loop awaits each uploadPart before the next ? confirm no concurrency.
3. ABORT/cleanup: on a thrown part, or signal.aborted, is the server partial DELETED (uploadAbort) exactly once and the error rethrown? Does an init failure (before any upload_id) avoid a bogus DELETE? Is AbortError swallowed (treated as cancel, not error) in useUpload?
4. The raw PUT: body is a Blob with content-type application/octet-stream + the cid header + signal. Is that correct vs request() (which forces application/json)? Any header/content-type issue that would break FastAPI's request.stream()?
5. useUpload state machine: empty file -> invalidUri before network; progress pct clamped 0..100; on success setPhase idle THEN onComplete (so the engine phase takes over); stale-run guard (a superseding start aborts the prior). Any setState-after-unmount risk? Be precise (file:line). Empty findings if clean.

2. assistant (2026-06-21T16:31:40.516Z)

I'll review T4's upload implementation rigorously. Let me start by reading the relevant files.

3. user (2026-06-21T16:31:41.462Z)

```
1      // Typed client for the engine FastAPI. SAME-ORIGIN relative `/api/*` (the appliance  
serves  
2      // UI + engine behind one origin; in dev Vite proxies `/api` ? :8000). Every call
```

```

carries a
3      // correlation id (header + logs) so a journey is traceable end to end.
4      import { log, newCorrelationId } from "../lib/log";
5      import type {
6          RallySegment,
7          QueryResponse,
8          CompileResponse,
9          EngineConfig,
10         ProcessResponse,
11         JobStatus,
12         UploadInitResponse,
13         UploadPartResponse,
14         UploadCompleteResponse,
15     } from "./types";
16
17     const BASE = "/api";
18
19     export class ApiError extends Error {
20         readonly cid?: string;
21         readonly status?: number;
22         readonly body?: string;
23         constructor(message: string, opts: { cid?: string; status?: number; body?: string;
cause?: unknown } = {}) {
24             super(message);
25             this.name = "ApiError";
26             this.cid = opts.cid;
27             this.status = opts.status;
28             this.body = opts.body;
29             if (opts.cause !== undefined) (this as { cause?: unknown }).cause = opts.cause;
30         }
31     }
32
33     interface RequestOpts {
34         method?: string;
35         body?: string;
36         headers?: Record<string, string>;
37         /** Correlation id to thread through this call; auto-generated if absent. */
38         cid?: string;
39     }
40
41     async function request<T>(path: string, opts: RequestOpts = {}): Promise<T> {
42         const cid = opts.cid ?? newCorrelationId();
43         const url = `${BASE}${path}`;
44         const method = opts.method ?? "GET";
45         log("info", "api.request", { cid, method, url });
46
47         let res: Response;
48         try {
49             res = await fetch(url, {
50                 method,
51                 body: opts.body,
52                 headers: { "content-type": "application/json", "x-correlation-id": cid,
...(opts.headers ?? {}) },
53             });
54         } catch (err) {
55             // Loud failure, never silent (LOCAL_APPLIANCE_ARCHITECTURE.md §5).
56             log("error", "api.network_error", { cid, method, url, error: String(err) });
57             throw new ApiError(`Network error calling ${method} ${url}`, { cid, cause: err });
58         }
59
60         if (!res.ok) {
61             const body = await res.text().catch(() => "");
62             log("error", "api.http_error", { cid, method, url, status: res.status, body:
body.slice(0, 500) });
63             throw new ApiError(`${method} ${url} failed: ${res.status}`, { cid, status:

```

```

res.status, body });
64     }
65
66     const data = (await res.json()) as T;
67     log("info", "api.response", { cid, method, url, status: res.status });
68     return data;
69 }
70
71 /** Submit a video (a `gs://`/`gcs://`/`s3://` bucket URI, or a path on the engine host)
for
72 * processing. POST /api/process {video_path} ? 202 {job_id, ...} [main.py:260]. Fast-
fails 400
73 * (unsupported scheme / out-of-allowed-root) or 404 (missing LOCAL file) ? surfaced as
ApiError. */
74 export async function processVideo(videoPath: string, cid?: string):
Promise<ProcessResponse> {
75     return request<ProcessResponse>("/process", {
76         method: "POST",
77         body: JSON.stringify({ video_path: videoPath }),
78         cid,
79     });
80 }
81
82 /** Poll a submitted job. GET /api/jobs/{job_id} [main.py:303]. 404 ? unknown job_id. */
83 export async function getJob(jobId: string, cid?: string): Promise<JobStatus> {
84     return request<JobStatus>(`/jobs/${encodeURIComponent(jobId)}`, { method: "GET", cid
});
85 }
86
87 /** List a video's detected rallies. POST /api/query {video_id} ? {play_segments}
[main.py:331]. */
88 export async function listRallies(videoId: string, cid?: string):
Promise<RallySegment[]> {
89     const data = await request<QueryResponse>("/query", {
90         method: "POST",
91         body: JSON.stringify({ video_id: videoId }),
92         cid,
93     });
94     return data.play_segments ?? [];
95 }
96
97 /** Stitch selected rallies into one reel. POST /api/compile {video_id, segment_ids,
output_filename} [main.py:342]. */
98 export async function compileReel(
99     videoId: string,
100     segmentIds: number[],
101     outputFilename: string,
102     cid?: string,
103 ): Promise<CompileResponse> {
104     return request<CompileResponse>("/compile", {
105         method: "POST",
106         body: JSON.stringify({ video_id: videoId, segment_ids: segmentIds, output_filename:
outputFilename }),
107         cid,
108     });
109 }
110
111 /** The stream/download URL for a compiled reel. GET
/api/highlights/{video_id}/{filename} [main.py:307]. */
112 export function highlightUrl(videoId: string, filename: string): string {
113     return
`${BASE}/highlights/${encodeURIComponent(videoId)}/${encodeURIComponent(filename)}`;
114 }
115
116 /** Read engine config (for the stitching floor). GET /api/config [main.py:121]. */

```

```

117     export async function getConfig(cid?: string): Promise<EngineConfig> {
118         return request<EngineConfig>("/config", { method: "GET", cid });
119     }
120
121     /** The shot-count floor /api/compile silently enforces ? the UI must warn, not hide
[main.py:362]. */
122     export function minShotCount(cfg: EngineConfig): number | undefined {
123         return cfg.stitching?.min_shot_count;
124     }
125
126     // ?? chunked upload (backend/api/uploads.py) ?????????????????????????????????
127     // A browser uploads a LOCAL file in sequential parts ? max_part_mb (free-tier edge
proxies cap
128     // bodies ~100 MB), then `complete` returns a server-host `path` to feed straight into
/api/process.
129
130     /** Start a chunked upload. 400 = bad extension; 413 = over the size limit
[uploads.py:64]. */
131     export async function uploadInit(filename: string, totalSizeBytes: number, cid?:
string): Promise<UploadInitResponse> {
132         return request<UploadInitResponse>("/upload/init", {
133             method: "POST",
134             body: JSON.stringify({ filename, total_size_bytes: totalSizeBytes }),
135             cid,
136         });
137     }
138
139     /** Append ONE part (raw bytes ? not JSON). Parts are strictly sequential 0,1,2,?
[uploads.py:89]. */
140     export async function uploadPart(
141         uploadId: string,
142         index: number,
143         chunk: Blob,
144         cid?: string,
145         signal?: AbortSignal,
146     ): Promise<UploadPartResponse> {
147         const c = cid ?? newCorrelationId();
148         const url = `${BASE}/upload/${encodeURIComponent(uploadId)}/part/${index}`;
149         log("info", "api.request", { cid: c, method: "PUT", url });
150         let res: Response;
151         try {
152             res = await fetch(url, {
153                 method: "PUT",
154                 body: chunk,
155                 headers: { "content-type": "application/octet-stream", "x-correlation-id": c },
156                 signal,
157             });
158         } catch (err) {
159             log("error", "api.network_error", { cid: c, method: "PUT", url, error: String(err)
});
160             throw new ApiError(`Network error calling PUT ${url}`, { cid: c, cause: err });
161         }
162         if (!res.ok) {
163             const body = await res.text().catch(() => "");
164             log("error", "api.http_error", { cid: c, method: "PUT", url, status: res.status,
body: body.slice(0, 500) });
165             throw new ApiError(`PUT ${url} failed: ${res.status}`, { cid: c, status: res.status,
body });
166         }
167         log("info", "api.response", { cid: c, method: "PUT", url, status: res.status });
168         return (await res.json()) as UploadPartResponse;
169     }
170
171     /** Finalize: returns {path, video_id} ? feed `path` to processVideo() [uploads.py:123].
*/

```

```

172     export async function uploadComplete(uploadId: string, totalParts: number, cid?:
string): Promise<UploadCompleteResponse> {
173         return
request<UploadCompleteResponse>(`/upload/${encodeURIComponent(uploadId)}/complete`, {
174             method: "POST",
175             body: JSON.stringify({ total_parts: totalParts }),
176             cid,
177         });
178     }
179
180     /** Abort an in-flight upload (best-effort cleanup of the server-side partial)
[uploads.py:147]. */
181     export async function uploadAbort(uploadId: string, cid?: string): Promise<void> {
182         await request<unknown>(`/upload/${encodeURIComponent(uploadId)}`, { method: "DELETE",
cid });
183     }
184
185     export interface UploadProgress {
186         sentBytes: number;
187         totalBytes: number;
188     }
189
190     /** Upload a local File end to end: init ? sequential parts (byte progress, abortable) ?
complete.
191     * On ANY failure or abort it best-effort DELETES the server-side partial, then
rethrows so the
192     * caller can classify it (the engine's 400/413 detail survives). Returns {path,
video_id}. */
193     export async function uploadFile(
194         file: File,
195         opts: { cid?: string; signal?: AbortSignal; onProgress?: (p: UploadProgress) => void }
= {},
196     ): Promise<UploadCompleteResponse> {
197         const cid = opts.cid ?? newCorrelationId();
198         const init = await uploadInit(file.name, file.size, cid);
199         const partBytes = Math.max(1, init.max_part_mb) * 1024 * 1024;
200         const totalParts = Math.max(1, Math.ceil(file.size / partBytes));
201         log("info", "upload.start", { cid, filename: file.name, size: file.size, parts:
totalParts, part_mb: init.max_part_mb });
202         let sent = 0;
203         try {
204             for (let i = 0; i < totalParts; i++) {
205                 if (opts.signal?.aborted) throw new DOMException("Upload aborted", "AbortError");
206                 const chunk = file.slice(i * partBytes, Math.min((i + 1) * partBytes, file.size));
207                 await uploadPart(init.upload_id, i, chunk, cid, opts.signal);
208                 sent += chunk.size;
209                 opts.onProgress?.({ sentBytes: sent, totalBytes: file.size });
210             }
211             const done = await uploadComplete(init.upload_id, totalParts, cid);
212             log("info", "upload.done", { cid, video_id: done.video_id, path: done.path, size:
done.size_bytes });
213             return done;
214         } catch (err) {
215             log("warn", "upload.aborting_partial", { cid, upload_id: init.upload_id, error:
String(err) });
216             await uploadAbort(init.upload_id, cid).catch(() => {});
217             throw err;
218         }
219     }
220

```

4. user (2026-06-21T16:31:41.945Z)

```

1      // Drives the in-UI file UPLOAD half of ingest (tracker T4): chunk a local file up to
the engine's

```

```

2      // /api/upload/* endpoints, report byte progress, then hand the returned server PATH to
the existing
3      // processing back-half (useIngestJob.submit). Alpha users have a FILE, not a gs:// URI
? this is the
4      // keystone that lets them point the tool at it from the browser.
5      //
6      // Kept i18n-FREE (returns stable IngestErrorCodes via classifyError, not prose) so it
unit-tests
7      // without a translation context ? IngestPanel maps codes to localized strings via
errorText. Abort
8      // is an AbortController: cancel/unmount/a superseding pick stops the in-flight upload
and the
9      // orchestrator best-effort DELETES the server-side partial.
10     import { useCallback, useEffect, useRef, useState } from "react";
11     import { uploadFile } from "../api/client";
12     import { classifyError, type IngestError } from "../lib/errors";
13     import { log, newCorrelationId } from "../lib/log";
14
15     export type UploadPhase =
16       | { status: "idle" }
17       | { status: "uploading"; pct: number; filename: string }
18       | { status: "error"; error: IngestError };
19
20     export function useUpload(onComplete: (path: string) => void) {
21       const [phase, setPhase] = useState<UploadPhase>({ status: "idle" });
22       const ctrlRef = useRef<AbortController | null>(null);
23       // Keep the latest onComplete without making start depend on it (a parent re-render
won't restart).
24       const onCompleteRef = useRef(onComplete);
25       onCompleteRef.current = onComplete;
26
27       const stop = useCallback(() => {
28         ctrlRef.current?.abort();
29         ctrlRef.current = null;
30       }, []);
31
32       // Abort any in-flight upload on unmount.
33       useEffect(() => stop, [stop]);
34
35       const start = useCallback(
36         (file: File) => {
37           // A 0-byte "video" is never valid ? reject before touching the network (reuse
invalidUri copy).
38           if (!file || file.size === 0) {
39             setPhase({ status: "error", error: { code: "invalidUri" } });
40             return;
41           }
42
43           stop(); // supersede any prior upload
44           const ctrl = new AbortController();
45           ctrlRef.current = ctrl;
46           const cid = newCorrelationId();
47           setPhase({ status: "uploading", pct: 0, filename: file.name });
48           log("info", "upload.submit_start", { cid, filename: file.name, size: file.size });
49
50           void (async () => {
51             try {
52               const res = await uploadFile(file, {
53                 cid,
54                 signal: ctrl.signal,
55                 onProgress: ({ sentBytes, totalBytes }) => {
56                   if (ctrl.signal.aborted) return;
57                   const pct = totalBytes > 0 ? Math.min(100, Math.round((sentBytes /
totalBytes) * 100)) : 0;
58                   setPhase({ status: "uploading", pct, filename: file.name });

```

```

59         },
60     });
61     if (ctrl.signal.aborted) return;
62     setPhase({ status: "idle" }); // hand off ? the engine processing phase takes
over
63     onCompleteRef.current(res.path);
64     } catch (err) {
65         // User-cancelled (abort) is silent, not an error.
66         if (ctrl.signal.aborted || (err instanceof DOMException && err.name ===
"AbortError")) return;
67         setPhase({ status: "error", error: classifyError(err) });
68         log("error", "upload.failed", { cid, filename: file.name, error: String(err)
});
69     }
70     })();
71     },
72     [stop],
73 );
74
75     const cancel = useCallback(() => {
76         stop();
77         setPhase({ status: "idle" });
78         log("info", "upload.cancel", {});
79     }, [stop]);
80
81     const reset = useCallback(() => {
82         stop();
83         setPhase({ status: "idle" });
84     }, [stop]);
85
86     const busy = phase.status === "uploading";
87     return { phase, start, cancel, reset, busy };
88 }
89

```

5. user (2026-06-21T16:31:42.412Z)

```

1     // Types mirror the engine's verified contract (badminton-highlight-indexer).
2     // Anchors: backend/infrastructure/database.py:124-135 (play_segments),
3     // backend/main.py:331-396 (query/compile), :307-328 (highlights), :121 (config).
4
5     export interface RallyMetadata {
6         /** Gemini's shuttle_exchange_count (fallback max(3, dur/0.8)); absent on some
detectors. */
7         shot_count?: number;
8         /** Gemini confidence STRING (often "Unknown") ? not a numeric meter. */
9         shot_count_confidence?: string;
10        detector?: string;
11        [key: string]: unknown;
12    }
13
14    export interface RallySegment {
15        /** The integer the UI selects and passes to /api/compile as segment_ids. */
16        id: number;
17        start_time: number;
18        end_time: number;
19        duration: number;
20        /** motion-segmenter FABRICATION (random) ? DO NOT surface (see
PER_RALLY_SELECTION.md). */
21        server: string | null;
22        /** motion-segmenter FABRICATION (random) ? DO NOT surface. */
23        receiver: string | null;
24        metadata: RallyMetadata;
25        /** motion-only fabricated sub-events ? DO NOT surface. */
26        events?: unknown[];

```

```

27     }
28
29     export interface QueryResponse {
30         play_segments: RallySegment[];
31     }
32
33     export interface CompileResponse {
34         status: string;
35         /** server-local path (NOT a URL) ? build the download URL with highlightUrl(). */
36         filepath: string;
37     }
38
39     export interface EngineConfig {
40         stitching?: {
41             /** The shot-count floor /api/compile silently enforces ? surface it, don't hide it.
42             */
43             min_shot_count?: number;
44             [key: string]: unknown;
45         };
46         [key: string]: unknown;
47     }
48     /** POST /api/process ? 202 [backend/main.py:260]. The job is queued; poll GET
49     /api/jobs/{job_id}. */
50     export interface ProcessResponse {
51         job_id: string;
52         /** Always "queued" at submit. */
53         status: string;
54         /** Relative poll path, e.g. "/api/jobs/<id>". */
55         poll: string;
56     }
57     /** Chunked upload (backend/api/uploads.py). The browser uploads a local file in
58     sequential parts
59     * ? max_part_mb, then `complete` returns a server-host `path` to feed straight into
60     POST /api/process. */
61     export interface UploadInitResponse {
62         upload_id: string;
63         /** Per-part ceiling in MB ? slice the file into parts this big. */
64         max_part_mb: number;
65         next_part: string;
66     }
67     export interface UploadPartResponse {
68         upload_id: string;
69         received_parts: number;
70         received_bytes: number;
71     }
72     export interface UploadCompleteResponse {
73         /** Absolute path on the engine host ? pass as video_path to POST /api/process. */
74         path: string;
75         video_id: string;
76         size_bytes: number;
77     }
78     /** GET /api/jobs/{job_id} [backend/main.py:303]. Two status layers:
79     * `status` = the EXECUTION state; `result.status` = the pipeline VERDICT
80     (success|failed).
81     * A `done` job whose `result.status` === "failed" (or with no video_id) is a success-
82     shaped
83     * FAILURE ? the UI must not treat it as a load. */
84     export interface JobStatus {
85         job_id: string;
86         status: "queued" | "running" | "done" | "failed";
87         /** Free-text coarse phase ("hashing", "segmenting (gemini)", "finished") ? NOT a
88         percentage. */

```



```

85     progress?: string | null;
86     /** The canonical video_id, surfaced top-level on success (also in result.video_id).
87     */
88     video_id?: string | null;
89     /** Execution error (worker crash / unexpected) when status === "failed". */
90     error?: string | null;
91     result?: { status?: string; video_id?: string; message?: string } | null;
92 }

```

6. user (2026-06-21T16:31:42.924Z)

```

1     """Chunked upload (B2, PR-2) ? extracted from backend.main into its own router.
2
3     Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
4     proxies cap request bodies at ~100 MB ? HOSTING_PLAN $2), are assembled server-side
5     under security.upload_dir, then submitted to /api/process by the returned path.
6     Upload state is in-process by design (this is the single-box alpha): a server restart
7     aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir
8     arrives with PR-3 identity scoping.
9
10    The startup config lives on ``backend.main.global_config`` (set ONLY in ``main()``).
11    ``_upload_cfg`` resolves it through the ``backend.main`` module object so it always
12    reads the live value the server set at startup (and so tests that monkeypatch
13    ``backend.main.global_config`` flow through unchanged).
14    """
15    import os
16    import threading
17    import uuid
18    from typing import Any, Dict, Optional
19
20    from fastapi import APIRouter, HTTPException, Request
21    from pydantic import BaseModel
22
23    from backend.utils.hashing import compute_video_id # canonical file-identity hashing
24    from backend.utils.paths import safe_output_filename
25
26    router = APIRouter()
27
28    _uploads: Dict[str, Dict[str, Any]] = {}
29    _uploads_lock = threading.Lock()
30
31
32    class UploadInitRequest(BaseModel):
33        filename: str
34        total_size_bytes: Optional[int] = None
35
36
37    class UploadCompleteRequest(BaseModel):
38        total_parts: int
39
40
41    def _upload_cfg():
42        # Resolved lazily (import inside the function) to read the LIVE startup config that
43        # main() sets on the module global, and to avoid a circular import at module load
44        # (main.py imports this router; this module must not import main at import time).
45        import backend.main as _main
46        sec = getattr(_main.global_config, "security", None)
47        upload_dir = getattr(sec, "upload_dir", None) or "output/uploads"
48        max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
49        part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
50        exts = [e.lower().lstrip(".") for e in (getattr(sec, "allowed_upload_extensions",
51        None) or ["mp4", "mov"])]
52        return upload_dir, max_bytes, part_bytes, exts

```

```

53
54 def _abort_upload(upload_id: str) -> None:
55     with _uploads_lock:
56         st = _uploads.pop(upload_id, None)
57     if st:
58         try:
59             os.remove(st["tmp_path"])
60         except OSError:
61             pass
62
63
64 @router.post("/api/upload/init")
65 def upload_init(req: UploadInitRequest):
66     """Start a chunked upload. Returns upload_id; PUT sequential parts next."""
67     upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
68     try:
69         name = safe_output_filename(req.filename)
70     except ValueError as e:
71         raise HTTPException(status_code=400, detail=str(e)) from e
72     ext = os.path.splitext(name)[1].lower().lstrip(".")
73     if ext not in exts:
74         raise HTTPException(status_code=400,
75                             detail=f"Extension '{ext}' not allowed. Allowed:
{sorted(exts)}")
76     if req.total_size_bytes is not None and req.total_size_bytes > max_bytes:
77         raise HTTPException(status_code=413,
78                             detail=f"File exceeds the {max_bytes // (1024 * 1024)} MB
upload limit.")
79     os.makedirs(upload_dir, exist_ok=True)
80     upload_id = uuid.uuid4().hex
81     tmp_path = os.path.join(upload_dir, f".partial-{upload_id}")
82     open(tmp_path, "wb").close()
83     with _uploads_lock:
84         _uploads[upload_id] = {"filename": name, "tmp_path": tmp_path, "parts": 0,
"bytes": 0}
85     return {"upload_id": upload_id, "max_part_mb": part_bytes // (1024 * 1024),
86           "next_part": f"/api/upload/{upload_id}/part/0"}
87
88
89 @router.put("/api/upload/{upload_id}/part/{index}")
90 async def upload_part(upload_id: str, index: int, request: Request):
91     """Append one part (raw bytes body). Parts are strictly sequential (0,1,2,...)."""
92     _, max_bytes, part_bytes, _ = _upload_cfg()
93     with _uploads_lock:
94         st = _uploads.get(upload_id)
95     if st is None:
96         raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
97     if index != st["parts"]:
98         raise HTTPException(status_code=409,
99                             detail=f"Out-of-order part: expected {st['parts']}, got
{index} (parts are sequential)")
100
101     received = 0
102     overflow = None
103     with open(st["tmp_path"], "ab") as fh:
104         async for chunk in request.stream():
105             received += len(chunk)
106             if received > part_bytes:
107                 overflow = f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part
limit."
108                 break
109             if st["bytes"] + received > max_bytes:
110                 overflow = f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total
limit."
111                 break

```

```

112         fh.write(chunk)
113     if overflow:
114         _abort_upload(upload_id)
115         raise HTTPException(status_code=413, detail=overflow + " Upload aborted ? re-
init to retry.")
116
117     with _uploads_lock:
118         st["parts"] += 1
119         st["bytes"] += received
120     return {"upload_id": upload_id, "received_parts": st["parts"], "received_bytes":
st["bytes"]}
121
122
123     @router.post("/api/upload/{upload_id}/complete")
124     def upload_complete(upload_id: str, req: UploadCompleteRequest):
125         """Finalize: verify part count, move into place, return {path, video_id} for
/api/process."""
126         upload_dir, _, _, _ = _upload_cfg()
127         with _uploads_lock:
128             st = _uploads.get(upload_id)
129             if st is None:
130                 raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
131             if st["parts"] == 0 or st["parts"] != req.total_parts:
132                 raise HTTPException(status_code=409,
133                                     detail=f"Expected {req.total_parts} part(s), received
{st['parts']}.")
134             final_path = os.path.join(upload_dir, st["filename"])
135             if os.path.exists(final_path): # never overwrite an earlier upload with the same
name
136                 stem, ext = os.path.splitext(st["filename"])
137                 final_path = os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
138             os.replace(st["tmp_path"], final_path)
139             with _uploads_lock:
140                 _uploads.pop(upload_id, None)
141             video_id = compute_video_id(final_path)
142             return {"path": os.path.abspath(final_path), "video_id": video_id,
143                     "size_bytes": os.path.getsize(final_path),
144                     "next": "POST /api/process with this path"}
145
146
147     @router.delete("/api/upload/{upload_id}")
148     def upload_abort(upload_id: str):
149         """Abort an in-flight upload and delete its partial file."""
150         with _uploads_lock:
151             known = upload_id in _uploads
152             if not known:
153                 raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
154             _abort_upload(upload_id)
155             return {"status": "aborted", "upload_id": upload_id}
156

```

7. assistant (2026-06-21T16:31:47.019Z)

Now let me verify the engine contract details and check the IngestPanel and tests for additional context.

8. user (2026-06-21T16:31:48.130Z)

```

1     // The front-half ingest screen (appliance S1/S2; tracker B-P2 + T4). Two ways to point
the tool at a
2     // video: UPLOAD a local file (primary ? alpha users have a file, not a URI; chunked to
the engine's
3     // /api/upload/* endpoints) or paste a `gs://` bucket URI / host path (secondary). Both
converge on
4     // the SAME back-half: POST /api/process ? poll GET /api/jobs/{id} ? hand the video_id

```

```

to App. Honest
5      // by construction: the engine reports coarse phase WORDS (not a %), and a success-
shaped failure
6      // (done but no rallies) is shown, never silently treated as a load.
7      import { useState } from "react";
8      import type { ChangeEvent, FormEvent } from "react";
9      import { useTranslation } from "react-i18next";
10     import { useIngestJob } from "../hooks/useIngestJob";
11     import { useUpload } from "../hooks/useUpload";
12     import { errorText } from "../lib/errorText";
13     import { StatusChip, Spinner } from "../StatusChip";
14
15     interface Props {
16       /** Called with the engine's video_id once a submitted job finishes successfully. */
17       onLoaded: (videoId: string) => void;
18     }
19
20     export function IngestPanel({ onLoaded }: Props) {
21       const { t } = useTranslation();
22       const [uri, setUri] = useState("");
23       const { phase, submit, cancel: cancelIngest, reset: resetIngest, busy: ingestBusy } =
useIngestJob(onLoaded);
24       // On upload completion, feed the returned server PATH straight into the engine
processing back-half.
25       const upload = useUpload((path) => submit(path));
26       const busy = ingestBusy || upload.busy;
27
28       const onSubmitUri = (e: FormEvent) => {
29         e.preventDefault();
30         submit(uri);
31       };
32       const onFile = (e: ChangeEvent<HTMLInputElement>) => {
33         const file = e.target.files?.[0];
34         if (file) upload.start(file);
35         e.target.value = ""; // let re-picking the same file fire change again
36       };
37       const cancelAll = () => {
38         upload.cancel();
39         cancelIngest();
40       };
41       const resetAll = () => {
42         upload.reset();
43         resetIngest();
44       };
45
46       // Shared mapping with the build flow (lib/errorText): an `engine` failure carries the
engine's own
47       // message (a 400 scheme/path/extension detail, a worker error); every other code maps
to a friendly
48       // localized line. Surface whichever half failed (upload or engine) ? they're mutually
exclusive.
49       const errorState = upload.phase.status === "error" ? upload.phase.error : phase.status
=== "error" ? phase.error : null;
50       const errorMsg = errorState ? errorText(t, errorState) : null;
51
52       return (
53         <section className="mt-6 rounded-2xl border border-black/10 bg-white p-5" data-
testid="ingest-panel">
54           <h2 className="text-lg font-bold">{t("ingest.title")}</h2>
55           <p className="mt-1 max-w-2xl text-sm text-ink/60">{t("ingest.intro")}</p>
56
57           {/* PRIMARY: upload a local file (T4). A native file input, visually a button
(?44px, A4). */}
58           <div className="mt-4">
59             <label

```

```

60         className={
61             "inline-flex min-h-11 items-center rounded-xl bg-ink px-4 py-2 text-sm font-
semibold text-white hover:bg-ink2 " +
62             (busy ? "pointer-events-none opacity-40" : "cursor-pointer")
63         }
64     >
65         {t("ingest.chooseFile")}
66         <input
67             type="file"
68             accept=".mp4, .mov, video/mp4, video/quicktime"
69             aria-label={t("ingest.uploadAria")}
70             disabled={busy}
71             onChange={onFile}
72             className="sr-only"
73         />
74     </label>
75     <p className="mt-2 text-xs text-ink/40">{t("ingest.fileHint")}</p>
76 </div>
77
78     {/* SECONDARY: or point at a cloud bucket / host path. */}
79     <div className="mt-4 border-t border-black/5 pt-4">
80         <p className="text-xs font-semibold uppercase tracking-wide text-
ink/40">{t("ingest.orLink")}</p>
81         <form onSubmit={onSubmitUri} className="mt-2 flex flex-wrap items-center gap-2">
82             <input
83                 type="text"
84                 value={uri}
85                 onChange={(e) => setUri(e.target.value)}
86                 placeholder={t("ingest.uriPlaceholder")}
87                 aria-label={t("ingest.uriAriaLabel")}
88                 disabled={busy}
89                 className="min-h-11 min-w-[18rem] flex-1 rounded-xl border border-black/15
bg-white px-4 py-2 text-sm text-ink outline-none focus:border-green disabled:opacity-50"
90             />
91             <button
92                 type="submit"
93                 disabled={busy}
94                 className="min-h-11 rounded-xl bg-ink px-4 py-2 text-sm font-semibold text-
white disabled:opacity-40"
95             >
96                 {phase.status === "submitting" ? t("ingest.submitting") :
t("ingest.submit")}
97             </button>
98         </form>
99         <p className="mt-2 text-xs text-ink/40">{t("ingest.hint")}</p>
100     </div>
101
102     {/* Live status (aria-live so screen readers hear progress) as icon+text chips ?
never icon-only
103         (A8). Upload shows a determinate % bar; the engine phase is indeterminate
(phase words). */}
104     <div aria-live="polite" className="mt-3 text-sm" data-testid="ingest-status">
105         {upload.phase.status === "uploading" && (
106             <div data-testid="upload-progress">
107                 <StatusChip tone="info" icon={<Spinner />} label={t("ingest.uploading", {
pct: upload.phase.pct })} />
108             <div
109                 className="mt-2 h-1.5 w-full max-w-sm overflow-hidden rounded-full bg-
black/10"
110                 role="progressbar"
111                 aria-valuenow={upload.phase.pct}
112                 aria-valuemin={0}
113                 aria-valuemax={100}
114             >
115                 <div className="h-full bg-green transition-all" style={{ width:

```

```

`${upload.phase.pct}%` }} />
116         </div>
117         <p className="mt-2 text-xs text-ink/50">{t("ingest.reassure")}</p>
118         <button
119             type="button"
120             onClick={cancelAll}
121             className="mt-2 inline-flex min-h-11 items-center rounded-xl border
border-black/15 px-4 text-sm hover:border-green"
122             >
123             {t("ingest.cancel")}
124         </button>
125     </div>
126 )}
127 {ingestBusy && (
128     <div>
129         <StatusChip
130             tone="info"
131             icon={<Spinner />}
132             label={
133                 t("ingest.working") +
134                 (phase.status === "polling" && phase.progress
135                     ? ` · ${t("ingest.progress", { progress: phase.progress })}`
136                     : "")
137             }
138         />
139         {/* Reassurance for slow uplinks: the job runs on the server, so it's safe
to wait. */}
140         <p className="mt-2 text-xs text-ink/50">{t("ingest.reassure")}</p>
141         <button
142             type="button"
143             onClick={cancelAll}
144             className="mt-2 inline-flex min-h-11 items-center rounded-xl border
border-black/15 px-4 text-sm hover:border-green"
145             >
146             {t("ingest.cancel")}
147         </button>
148     </div>
149 )}
150 {errorMsg && (
151     <div className="rounded-lg border border-red-200 bg-red-50 px-3 py-2 text-
red-800" data-testid="ingest-error">
152         <StatusChip tone="error" icon="?" label={t("ingest.statusFailed")} />
153         <p className="mt-2">{errorMsg}</p>
154         <button
155             type="button"
156             onClick={resetAll}
157             className="mt-2 inline-flex min-h-11 items-center rounded-lg border
border-red-300 px-3 text-sm hover:bg-red-100"
158             >
159             {t("ingest.retry")}
160         </button>
161     </div>
162 )}
163 </div>
164 </section>
165 );
166 }
167

```

9. user (2026-06-21T16:31:48.654Z)

```

1      // Error helpers shared by the ingest flow and the build flow. Kept i18n-FREE: callers
map an
2      // IngestErrorCode to a localized string (so hooks stay testable without a translation
context),

```

```

3      // while the engine's own `detail` text passes through verbatim (it's the most specific
message).
4      import { ApiError } from "../api/client";
5
6      /** Pull FastAPI's {"detail": "..."} message out of an error body, else return the raw
body. */
7      export function parseDetail(body: string): string {
8          try {
9              const j = JSON.parse(body) as { detail?: unknown };
10             if (j && typeof j.detail === "string") return j.detail;
11         } catch {
12             /* not JSON ? fall through to the raw body */
13         }
14         return body;
15     }
16
17     /** Stable, localizable failure categories surfaced by the ingest flow.
18     * `engine` carries the engine's own message in `detail`; the rest are localized by
code. */
19     export type IngestErrorCode = "invalidUri" | "networkError" | "emptyResult" | "timeout"
| "engine";
20
21     export interface IngestError {
22         code: IngestErrorCode;
23         /** Present for `engine` ? the verbatim FastAPI detail / status; rendered as-is, not
translated. */
24         detail?: string;
25     }
26
27     /** Classify a thrown client error into a stable code. A network throw (ApiError with no
`status`)
28     * is the common "engine unreachable" case ? its own friendly code; any HTTP error
surfaces the
29     * engine's `detail` (e.g. an unsupported-scheme 400, a not-found 404). */
30     export function classifyError(err: unknown): IngestError {
31         if (err instanceof ApiError) {
32             if (err.status === undefined) return { code: "networkError" }; // fetch threw ?
engine truly unreachable
33             if (err.body) {
34                 const detail = parseDetail(err.body);
35                 if (detail) return { code: "engine", detail }; // a real engine message (e.g. a
400 unsupported-scheme)
36             }
37             // A bare 5xx with no body is almost always the engine being down / a gateway that
can't reach it
38             // (e.g. the dev proxy when :8000 is off) ? for a non-technical user a friendly "is
it running?"
39             // beats a cryptic "HTTP 500". A 4xx with no body keeps the status (it's a client-
side mistake).
40             if (err.status >= 500) return { code: "networkError" };
41             return { code: "engine", detail: `HTTP ${err.status}` };
42         }
43         return { code: "engine", detail: String(err) };
44     }
45

```

10. assistant (2026-06-21T16:31:53.533Z)

Now I have the full picture. Let me work through the five rigorous checks systematically. Let me verify a few specific edge cases against the engine contract.

Let me check the chunking math edge cases carefully and the `next\_part` type mismatch I noticed.

The engine returns `next\_part: "/api/upload/{upload\_id}/part/0"` (a string), and `types.ts` line 63 declares `next\_part: string`. That matches. Good.

Let me verify the test file to understand what's already covered.

11. user (2026-06-21T16:31:54.648Z)

```
web\src\api\upload.test.ts
web\src\components\upload.test.tsx
```

12. user (2026-06-21T16:31:54.674Z)

```
web\src\api\client.test.ts
web\src\lib\selection.test.ts
web\src\lib\query.test.ts
web\src\lib\floor.test.ts
web\src\lib\format.test.ts
web\src\components\QueryBar.examples.test.ts
web\src\i18n\i18n.test.tsx
web\src\hooks\useIngestJob.test.ts
web\src\i18n\hygiene.test.ts
web\src\lib\errors.test.ts
web\src\components\IngestPanel.test.tsx
web\src\components\A4.touch.test.tsx
web\src\components\build-error.test.tsx
web\src\lib\errorText.test.ts
web\src\components\A8.status.test.tsx
web\src\api\upload.test.ts
web\src\hooks\useUpload.test.ts
web\src\components\upload.test.tsx
```

13. assistant (2026-06-21T16:31:57.964Z)

Let me read the upload tests to see what edge cases are already covered, so I don't report false positives on things that are tested.

14. user (2026-06-21T16:31:58.945Z)

```
1      // Chunked-upload client (T4): init ? sequential PUT parts (byte progress) ? complete,
with a
2      // best-effort DELETE of the server-side partial on any failure. Pins the part slicing
(sizes +
3      // strict 0,1,2 order), the progress callback, and that a mid-upload failure cleans up +
rethrows.
4      import { describe, it, expect, vi, afterEach } from "vitest";
5      import { uploadFile, ApiError } from "../client";
6
7      type FetchFn = typeof fetch;
8      const MB = 1024 * 1024;
9      const ok = (j: unknown) => ({ ok: true, status: 200, json: async () => j, text: async ()
=> JSON.stringify(j) });
10     const httpErr = (status: number, detail: string) => ({
11       ok: false, status, json: async () => ({ detail }, text: async () => JSON.stringify({
detail },
12     }));
13
14     afterEach(() => {
15       vi.restoreAllMocks();
16       vi.unstubAllGlobals();
17     });
18
19     /** A routing fetch over the /api/upload/* surface; records the order + sizes of the
calls. */
20     function router(opts: { partFailsAt?: number; initStatus?: number } = {}) {
21       const calls = { init: 0, parts: [] as number[], partSizes: [] as number[], complete:
null as number | null, deletes: 0 };
22       const f = vi.fn(async (url: string | URL, init?: RequestInit) => {
```



```

23     const u = String(url);
24     const m = init?.method ?? "GET";
25     if (u === "/api/upload/init" && m === "POST") {
26         calls.init += 1;
27         if (opts.initStatus) return httpErr(opts.initStatus, "Extension '.txt' not
allowed");
28         return ok({ upload_id: "up1", max_part_mb: 1, next_part: "/api/upload/up1/part/0"
});
29     }
30     const part = u.match(/^\/api\/upload\/up1\/part\/(\d+)\$/);
31     if (part && m === "PUT") {
32         const idx = Number(part[1]);
33         if (opts.partFailsAt === idx) return httpErr(413, "Part exceeds the per-part
limit");
34         const size = (init?.body as Blob).size;
35         calls.parts.push(idx);
36         calls.partSizes.push(size);
37         return ok({ upload_id: "up1", received_parts: calls.parts.length, received_bytes:
size });
38     }
39     if (u === "/api/upload/up1/complete" && m === "POST") {
40         calls.complete = JSON.parse(init!.body as string).total_parts as number;
41         return ok({ path: "/srv/uploads/match.mp4", video_id: "VID", size_bytes: 5 * MB
});
42     }
43     if (u === "/api/upload/up1" && m === "DELETE") {
44         calls.deletes += 1;
45         return ok({ status: "aborted", upload_id: "up1" });
46     }
47     throw new Error(`unexpected fetch ${m} ${u}`);
48 } as unknown as FetchFn;
49 return { f, calls };
50 }
51
52 const videoFile = (bytes: number, name = "match.mp4") =>
53     new File([new Uint8Array(bytes)], name, { type: "video/mp4" });
54
55 describe("uploadFile", () => {
56     it("slices into sequential parts ? max_part_mb, completes, and returns {path,
video_id}", async () => {
57         const { f, calls } = router();
58         vi.stubGlobal("fetch", f);
59         const progress: number[] = [];
60
61         const res = await uploadFile(videoFile(2.5 * MB), { onProgress: (p) =>
progress.push(p.sentBytes) });
62
63         expect(calls.parts).toEqual([0, 1, 2]); // 2.5MB / 1MB = 3 strictly-sequential parts
64         expect(calls.partSizes).toEqual([1 * MB, 1 * MB, 0.5 * MB]); // last part is the
remainder
65         expect(calls.complete).toBe(3); // total_parts handed to complete
66         expect(progress[progress.length - 1]).toBe(2.5 * MB); // progress reaches 100%
67         expect(res).toMatchObject({ path: "/srv/uploads/match.mp4", video_id: "VID" });
68     });
69
70     it("a single small file uploads as ONE part", async () => {
71         const { f, calls } = router();
72         vi.stubGlobal("fetch", f);
73         await uploadFile(videoFile(0.3 * MB));
74         expect(calls.parts).toEqual([0]);
75         expect(calls.complete).toBe(1);
76     });
77
78     it("on a part failure it DELETES the server-side partial and rethrows the engine's
reason", async () => {

```

```

79     const { f, calls } = router({ partFailsAt: 1 });
80     vi.stubGlobal("fetch", f);
81     await expect(uploadFile(videoFile(2.5 * MB))).rejects.toMatchObject({ name:
"ApiError", status: 413 });
82     expect(calls.parts).toEqual([0]); // stopped at the failing part
83     expect(calls.deletes).toBe(1); // partial cleaned up
84   });
85
86   it("an init rejection (bad extension) rethrows and never starts parts or a cleanup",
async () => {
87     const { f, calls } = router({ initStatus: 400 });
88     vi.stubGlobal("fetch", f);
89     await expect(uploadFile(videoFile(1 * MB,
"notes.txt"))).rejects.toBeInstanceOf(ApiError);
90     expect(calls.parts).toEqual([]);
91     expect(calls.deletes).toBe(0); // nothing to clean ? no upload_id yet
92   });
93 });
94

```

15. user (2026-06-21T16:31:59.027Z)

```

1    // @vitest-environment jsdom
2    //
3    // The upload state machine (T4): start(file) ? uploading(%) ? hand the server PATH to
onComplete |
4    // error. Pins: an empty file is rejected before any network call, a happy upload calls
onComplete
5    // with the returned path and returns to idle, and an engine rejection surfaces a
classified error.
6    import { describe, it, expect, vi, afterEach } from "vitest";
7    import { renderHook, act, waitFor } from "@testing-library/react";
8    import { useUpload } from "../useUpload";
9
10   type FetchFn = typeof fetch;
11   const MB = 1024 * 1024;
12   const ok = (j: unknown) => ({ ok: true, status: 200, json: async () => j, text: async ()
=> JSON.stringify(j) });
13   const httpErr = (status: number, detail: string) => ({
14     ok: false, status, json: async () => ({ detail }), text: async () => JSON.stringify({
detail }),
15   });
16
17   function uploadFetch(opts: { partFailsAt?: number } = {}): FetchFn {
18     return vi.fn(async (url: string | URL, init?: RequestInit) => {
19       const u = String(url);
20       const m = init?.method ?? "GET";
21       if (u === "/api/upload/init") return ok({ upload_id: "up1", max_part_mb: 1,
next_part: "/api/upload/up1/part/0" });
22       const part = u.match(/\/part\/(\d+)\$/);
23       if (part && m === "PUT") {
24         if (opts.partFailsAt === Number(part[1])) return httpErr(413, "Part exceeds the
per-part limit");
25         return ok({ upload_id: "up1", received_parts: 1, received_bytes: (init?.body as
Blob).size });
26       }
27       if (u.endsWith("/complete")) return ok({ path: "/srv/uploads/match.mp4", video_id:
"VID", size_bytes: 1 });
28       if (m === "DELETE") return ok({ status: "aborted" });
29       throw new Error(`unexpected ${m} ${u}`);
30     }) as unknown as FetchFn;
31   }
32
33   const videoFile = (bytes: number, name = "match.mp4") => new File([new
Uint8Array(bytes)], name, { type: "video/mp4" });

```

```

34
35   afterEach(() => {
36     vi.restoreAllMocks();
37     vi.unstubAllGlobals();
38   });
39
40   describe("useUpload", () => {
41     it("rejects an empty file before any network call", () => {
42       const f = vi.fn();
43       vi.stubGlobal("fetch", f as unknown as FetchFn);
44       const onComplete = vi.fn();
45       const { result } = renderHook(() => useUpload(onComplete));
46
47       act(() => result.current.start(videoFile(0)));
48
49       expect(result.current.phase).toEqual({ status: "error", error: { code: "invalidUri"
50     } } });
51       expect(f).not.toHaveBeenCalled();
52       expect(onComplete).not.toHaveBeenCalled();
53     });
54     it("uploads a file and hands the returned server path to onComplete", async () => {
55       vi.stubGlobal("fetch", uploadFetch());
56       const onComplete = vi.fn();
57       const { result } = renderHook(() => useUpload(onComplete));
58
59       act(() => result.current.start(videoFile(0.3 * MB)));
60       await waitFor(() =>
61         expect(onComplete).toHaveBeenCalled()
62         expect(result.current.phase.status).toBe("idle"); // handed off ? the engine phase
63         takes over
64       );
65     });
66     it("surfaces a classified error when a part is rejected", async () => {
67       vi.stubGlobal("fetch", uploadFetch({ partFailsAt: 0 }));
68       const onComplete = vi.fn();
69       const { result } = renderHook(() => useUpload(onComplete));
70
71       act(() => result.current.start(videoFile(0.3 * MB)));
72       await waitFor(() => expect(result.current.phase.status).toBe("error"));
73       expect(onComplete).not.toHaveBeenCalled();
74       // a 413 with a {detail} body classifies as an engine error (its message survives)
75       expect(result.current.phase).toMatchObject({ status: "error", error: { code:
76         "engine" } });
77     });
78   });
79 }

```

16. assistant (2026-06-21T16:32:12.760Z)

I now have a thorough understanding. Let me work through each of the five checks precisely, including the engine contract edge cases.

```

**Check 1 ? Chunking math** (`client.ts:199-210`):
- `partBytes = Math.max(1, init.max_part_mb) * 1024 * 1024` ? guards `max_part_mb` = 0/NaN?
  `Math.max(1, 0)` = 1, `Math.max(1, undefined)` = NaN. But TS types it as `number` and engine
  always returns ?1. The `Math.max(1, ...)` handles 0. NaN only if engine sends garbage. Minor.
- `totalParts = Math.max(1, Math.ceil(file.size / partBytes))` ? for size>0, `Math.ceil` ? 1
  already, but `Math.max(1, ...)` is belt-and-suspenders. Fine.
- Exact multiple: size = 2*partBytes ? `ceil(2)` = 2 parts. Loop i=0,1. i=0: slice(0,
  min(partBytes, 2pb)) = [0, pb]. i=1: slice(pb, min(2pb, 2pb)) = [pb, 2pb]. No trailing empty
  part. Correct.
- Last part remainder: 2.5MB/1MB ? ceil = 3. i=2: slice(2MB, min(3MB, 2.5MB)) = [2MB, 2.5MB] =
  0.5MB. Correct.

```

Chunking math is ****correct****. Now let me look for the real issues.

****Potential issue: empty-part / per-part overflow boundary.**** The engine (`uploads.py:106``) rejects when ``received > part_bytes`` (strict ``>``). The client sends parts of exactly ``partBytes`` (`= `max_part_mb * 1024 * 1024``), and the engine's ``part_bytes`` = ``max_part_mb * 1024 * 1024`` (same value, since engine returns ``part_bytes // (1024*1024)`` as ``max_part_mb``). So a full part of exactly ``partBytes`` gives ``received == part_bytes``, not ``>``. No false 413.

But there's a subtle truncation issue. Let me verify the round-trip.

The engine computes ``part_bytes = max_part_mb_config * 1024 * 1024`` (line 49), then returns ``max_part_mb = part_bytes // (1024*1024)`` (line 85). If config ``max_part_mb`` is an integer, round-trip is exact. The client multiplies back: ``init.max_part_mb * 1024 * 1024`` = same ``part_bytes``. So client part size == engine ``part_bytes`` exactly. A full part: ``received == part_bytes``, passes. Correct.

Let me double-check the ****total-size limit interaction**** and one concurrency subtlety in the abort path.

17. user (2026-06-21T16:32:13.895Z)

```
1      // @vitest-environment jsdom
2      //
3      // T4 end to end through IngestPanel: pick a local file ? it chunks up to /api/upload/*
? the returned
4      // server PATH is fed to POST /api/process ? poll ? onLoaded(video_id). The keystone for
alpha users
5      // who have a FILE, not a gs:// URI. Also pins that the file picker is present +
labelled (a11y/A4).
6      import { describe, it, expect, vi, beforeEach, afterEach } from "vitest";
7      import { render, screen, cleanup, fireEvent, act } from "@testing-library/react";
8      import i18n from "../i18n";
9      import { IngestPanel } from "../IngestPanel";
10
11     // Flush promise microtasks (the upload/process awaits) under fake timers ? waitFor
can't advance them.
12     async function flush(times = 15) {
13       for (let i = 0; i < times; i += 1) {
14         await act(async () => {
15           await vi.advanceTimersByTimeAsync(0);
16         });
17       }
18     }
19
20     const MB = 1024 * 1024;
21     const ok = (j: unknown) => ({ ok: true, status: 200, json: async () => j, text: async ()
=> JSON.stringify(j) });
22
23     let processBody: { video_path?: string } | null = null;
24
25     function uploadThenProcessFetch() {
26       processBody = null;
27       return vi.fn(async (url: string | URL, init?: RequestInit) => {
28         const u = String(url);
29         const m = init?.method ?? "GET";
30         if (u === "/api/upload/init") return ok({ upload_id: "up1", max_part_mb: 1,
next_part: "/api/upload/up1/part/0" });
31         if (u.includes("/api/upload/up1/part/")) return ok({ upload_id: "up1",
received_parts: 1, received_bytes: (init?.body as Blob).size });
32         if (u.endsWith("/complete")) return ok({ path: "/srv/uploads/match.mp4", video_id:
"PRE", size_bytes: 1 });
33         if (u === "/api/process" && m === "POST") {
34           processBody = JSON.parse(init!.body as string);
35           return ok({ job_id: "j", status: "queued", poll: "/api/jobs/j" });
36         }
```

```

37         if (u.startsWith("/api/jobs/")) return ok({ job_id: "j", status: "done", video_id:
"VIDFINAL", result: { status: "success", video_id: "VIDFINAL" } });
38         throw new Error(`unexpected fetch ${m} ${u}`);
39     }) as unknown as typeof fetch;
40 }
41
42     const videoFile = (bytes: number) => new File([new Uint8Array(bytes)], "match.mp4", {
type: "video/mp4" });
43
44     const pickFile = (file: File) => {
45         const input = screen.getByLabelText("Choose a video file to upload");
46         Object.defineProperty(input, "files", { value: [file], configurable: true });
47         fireEvent.change(input);
48     };
49
50     beforeEach(async () => {
51         window.localStorage.clear();
52         await i18n.changeLanguage("en");
53     });
54
55     afterEach(() => {
56         cleanup();
57         vi.restoreAllMocks();
58         vi.unstubAllGlobals();
59         vi.useRealTimers();
60     });
61
62     describe("T4: in-UI file upload", () => {
63         it("offers a labelled file picker (present for a keyboard/SR user)", () => {
64             vi.stubGlobal("fetch", vi.fn());
65             render(<IngestPanel onLoaded={vi.fn()} />);
66             expect(screen.getByLabelText("Choose a video file to upload")).toBeTruthy();
67             expect(screen.getByText("Choose a video file")).toBeTruthy();
68         });
69
70         it("uploads the picked file, feeds the server path to /api/process, and loads the
resulting video_id", async () => {
71             vi.useFakeTimers();
72             vi.stubGlobal("fetch", uploadThenProcessFetch());
73             const onLoaded = vi.fn();
74             render(<IngestPanel onLoaded={onLoaded} />);
75
76             pickFile(videoFile(0.3 * MB));
77
78             // Flush the upload chain (init ? part ? complete) + the process submit ? schedules
the poll.
79             await flush();
80             // The uploaded server PATH (not the URI) is what gets processed.
81             expect(processBody?.video_path).toBe("/srv/uploads/match.mp4");
82
83             // Poll tick ? done ? the job's canonical video_id is handed up.
84             await act(async () => {
85                 await vi.advanceTimersByTimeAsync(1000);
86             });
87             await flush();
88             expect(onLoaded).toHaveBeenCalledWith("VIDFINAL");
89         });
90     });
91

```

18. assistant (2026-06-21T16:32:36.988Z)

I now have full context. Let me work through the remaining checks precisely and identify the genuine findings.

****Check 2 ? Sequential order:**** The loop at ``client.ts:204-210`` ``await uploadPart(...)`` before incrementing ``i``. Strictly sequential, no ``Promise.all`/concurrency`. Parts go 0,1,2.
****Correct.**** Confirmed by ``upload.test.ts:63``.

****Check 3 ? Abort/cleanup:****

- On a thrown part or ``signal.aborted``, the ``catch`` at ``client.ts:214-218`` calls ``uploadAbort(...).catch(() => {})`` (best-effort, swallows DELETE errors) and rethrows ``err``. DELETE happens once. ****Correct.****
- Init failure: ``uploadInit`` is at line 198, ****outside**** the try block (try starts at 203). If init throws, we never reach the catch, so no bogus DELETE. ****Correct.**** Confirmed by ``upload.test.ts:86-92``.
- `AbortError` swallowed in ``useUpload``: line 66 ``if (ctrl.signal.aborted || (err instanceof DOMException && err.name === "AbortError")) return;``. ****Correct.****

****Check 4 ? Raw PUT:**** ``uploadPart`` (lines 152-157) uses ``fetch`` directly (not ``request()``), sets ``content-type: application/octet-stream``, the cid header, and ``signal``. It does NOT go through ``request()`` which would force ``application/json``. The body is a ``Blob``. FastAPI reads via ``request.stream()`` which is content-type agnostic. ****Correct.****

Now let me dig into the genuine issues I've found.

****Finding A ? Progress accounting double-counts / `sent` uses `chunk.size` not server-confirmed bytes.**** Minor at most ? ``chunk.size`` is exact. Not a bug.

****Finding B ? ``Math.max(1, init.max_part_mb)`` does NOT guard against ``NaN`/missing`.`**
``Math.max(1, NaN)`` = NaN ? ``partBytes`` = NaN ? ``Math.ceil(size/NaN)`` = NaN ? ``Math.max(1, NaN)`` = NaN ? loop ``i < NaN`` is false ? 0 parts ? ``uploadComplete(uploadId, NaN)``. Let me verify what the engine does with ``total_parts: NaN``. The prompt explicitly asks "What if ``max_part_mb`` is 0 or missing (``Math.max(1,...)`` guard)?" ? so this is in scope. ``Math.max(1, 0)`` = 1 (guarded), but ``Math.max(1, undefined)`` = NaN (NOT guarded). However, ``init.max_part_mb`` is typed ``number`` and the engine always returns it, so this is a defensive nit.

Let me verify the most important real concern: the ****total-size pre-check gap and the per-part vs. progress semantics****, plus whether there's a ****stale-run race**** in ``useUpload`` where progress from a superseded run can leak. Let me re-examine the superseding-start race carefully.

In ``useUpload.start``: ``stop()`` aborts the old controller (line 43), creates a new ``ctrl`` (line 44), stores it in ``ctrlRef`` (line 45). The old async IIFE's ``onProgress`` checks ``ctrl.signal.aborted`` (line 56) ? but ``ctrl`` here is ****captured by closure**** = the old controller, which was just aborted by ``stop()``. So old progress callbacks return early. The success path checks ``ctrl.signal.aborted`` (line 61) = old controller aborted ? returns.
****Correct ? no stale-run leak.****

Let me check one more genuine concern: ****setState-after-unmount****. On unmount, ``useEffect`` cleanup calls ``stop()`` ? aborts. The in-flight IIFE: when its ``await uploadFile`` rejects/resolves, line 61/66 checks ``ctrl.signal.aborted`` (true after abort) ? returns before any ``setPhase``. So no ``setState`` after unmount ****in the success and abort cases****. But what about the ****progress callback**** firing during the synchronous window ? covered by line 56. And the ****error path****: if abort triggers an `AbortError`, line 66 returns. Good. ****No setState-after-unmount.****

Now let me confirm the genuine findings around the engine total-size limit and a real edge case: the client never sends ``total_size_bytes``... actually it does (``client.ts:198`` passes ``file.size``). Good ? engine can 413 early.

Let me verify one real concern about the engine's ``safe_output_filename`` and whether the client's ``file.name`` could break, and check the per-part overflow exact-boundary once more by reading the config defaults.

19. user (2026-06-21T16:32:37.554Z)

```
1      """Path-safety helpers for the API boundary (untrusted request fields)."""
2      import os
3      from typing import Any, Optional
4
```

```

5
6     def output_base(config: Any) -> str:
7         """The configured base dir for a segmenter's LOCAL scratch/staging artifacts
8         (COMPUTE_DECOUPLED_SERVING M3). Returns ``output.output_dir`` (default ``"output"``)
so the
9         chunk-staging / AI-handoff / temp dirs follow ``--output-dir`` /
``output.output_dir`` instead of
10         a hardcoded ``./output``. The DEFAULT is ``"output"`` ? byte-identical to the prior
literal.
11
12         This is the LOCAL output dir, deliberately NOT the storage ``workspace_root`` (which
may be a
13         ``gs://`` URI you can't ``mkdirs``). Accepts the typed AppConfig or the legacy
dict-shim
14         (``.get("output", {}).get("output_dir")``); any miss or empty value falls back to
``"output"``. """
15         try:
16             out = config.get("output", {}).get("output_dir")
17         except Exception:
18             out = None
19         return out or "output"
20
21
22     def safe_output_filename(name: str) -> str:
23         """Validate an output filename is a BARE name (no dir part, no traversal, not
absolute).
24
25         ``os.path.join(output_dir, name)`` silently discards ``output_dir`` when ``name`` is
absolute,
26         and ``..`` segments escape it ? so an unchecked ``output_filename`` is an arbitrary-
write
27         primitive. Returns the name unchanged when safe; raises ValueError otherwise.
28         """
29         if not name or not name.strip():
30             raise ValueError("output_filename must not be empty")
31         if "\x00" in name:
32             raise ValueError("output_filename must not contain a null byte")
33         if "/" in name or "\\" in name:
34             raise ValueError(f"output_filename must not contain a path separator: {name!r}")
35         if os.path.isabs(name) or name != os.path.basename(name) or name in (".", ".."):
36             raise ValueError(f"output_filename must be a bare filename, got: {name!r}")
37         return name
38
39
40     def resolve_under_root(path: str, root: str) -> str:
41         """Return the real path, asserting it is contained under ``root`` (raises ValueError
if not). """
42         rp = os.path.realpath(path)
43         rr = os.path.realpath(root)
44         try:
45             contained = os.path.commonpath([rp, rr]) == rr
46         except ValueError: # different drives on Windows
47             contained = False
48         if not contained:
49             raise ValueError(f"path {path!r} escapes the allowed root {root!r}")
50         return rp
51
52
53     def validate_input_path(path: str, allowed_root: Optional[str] = None) -> None:
54         """Reject null bytes and (when an allowed_root is configured) out-of-root inputs.
55
56         With no allowed_root (default, single-user local mode) only the null-byte check
applies,
57         so the local 'point it at any of my videos' workflow is preserved. Setting an
58         allowed_root (a future hosted/tenant deployment) enforces containment.

```

```

59         """
60         if "\\x00" in path:
61             raise ValueError("video_path must not contain a null byte")
62         if allowed_root:
63             resolve_under_root(path, allowed_root)
64

```

20. assistant (2026-06-21T16:32:53.435Z)

This surfaces a genuine finding. ``safe_output_filename`` raises ``ValueError`` (? 400) if ``file.name`` contains a path separator, but on the client side ``file.name`` from a browser `<input type="file">` is always a bare filename ? browsers strip the path. So that's fine in practice. But there's a real issue worth noting: **a filename with characters the engine rejects but the browser allows** (e.g. a backslash on certain platforms is impossible from the picker). Not a real client bug.

Let me now finalize. Let me re-examine one subtle but real concern in the abort path: **``uploadPart`` is passed ``signal`` but if ``signal.aborted`` is already true *between* the ``for``-loop check and the fetch**, fetch with an already-aborted signal throws ``AbortError`` synchronously-ish ? caught at line 214 ? DELETE ? rethrow ? useUpload swallows. Fine.

And the **most genuine functional finding**: the per-iteration abort check at ``client.ts:205`` throws a ``DOMException`` `AbortError`, then the catch fires ``uploadAbort``. But if the abort happened because the **component unmounted**, the ``uploadAbort`` DELETE still fires (best-effort) ? that's correct and intended.

Let me verify there are no real correctness bugs in the progress ``pct`` clamping and the ``totalBytes > 0`` guard. Line 57: ``totalBytes > 0 ? Math.min(100, Math.round(...)) : 0``. Since empty files are rejected upstream (``file.size === 0`` ? line 38), ``totalBytes`` is always `> 0` here. Clamped to `?100`. **Correct.**

Now, the one **real bug** I want to confirm: the **``max_part_mb`` NaN/missing guard gap**. And let me check whether there's any issue with **``uploadComplete`` being skipped on abort** but the engine partial leaking ? no, abort DELETES it.

Let me also verify a genuinely important edge case the prompt flagged: **"What if size is an exact multiple of partBytes (no trailing empty part)?"** ? confirmed correct above (size=2pb ? 2 parts, no empty 3rd). The engine would 409 on a 0-byte trailing part anyway since ``complete`` checks ``parts == total_parts``, but the client never sends one. Good.

I've completed a rigorous review. The implementation is largely correct. Let me compile my findings ? there are a couple of genuine minor/nit items but no blockers or majors.

21. user (2026-06-21T16:33:15.594Z)

Output does not match required schema: root: must have required property 'findings'

22. assistant (2026-06-21T16:33:19.029Z)

The schema requires ``findings`` and ``summary`` at the top level of the tool input, not nested. Let me re-call with the correct shape.

23. user (2026-06-21T16:33:34.117Z)

Output does not match required schema: root: must have required property 'findings'

24. assistant (2026-06-21T16:33:37.776Z)

The ``findings`` parameter must be an actual JSON array, not a string. Let me pass it as a proper array structure.

25. user (2026-06-21T16:33:53.899Z)

Output does not match required schema: root: must have required property 'findings'

26. assistant (2026-06-21T16:33:59.061Z)

The XML-style parameter encoding may be mangling the array. Let me try passing the entire input as a single JSON object ? the harness likely expects the structured payload differently. Let me verify by reading the schema again: it requires `findings` (array) and `summary` (string) as top-level required properties. The issue is likely that the `findings` value isn't being parsed as an array from the multiline content. Let me put it on compact single-line JSON.

27. user (2026-06-21T16:34:08.151Z)

Output does not match required schema: root: must have required property 'findings'

28. assistant (2026-06-21T16:34:12.499Z)

The harness keeps reporting `findings` missing, which suggests the array passed via the XML parameter syntax isn't being decoded into the object. Let me try the alternative where parameters wrap differently ? passing both as a single JSON-encoded blob isn't the convention. Let me try making `findings` an empty array first to isolate whether it's the array content or the array itself that fails.

29. user (2026-06-21T16:34:12.988Z)

Structured output provided successfully

30. assistant (2026-06-21T16:34:17.956Z)

The empty array works ? so the issue was the array content/encoding in the larger payload. The schema accepted `[]`. The problem was likely a parsing issue with the multi-element array content (possibly the colons or special characters). Let me re-submit the real findings, keeping the JSON clean and on one line per element with no problematic characters.

31. user (2026-06-21T16:34:32.407Z)

Output does not match required schema: root: must have required property 'findings'

32. assistant (2026-06-21T16:34:38.490Z)

Interesting ? the empty array `[]` worked but two real elements fail. The difference must be the `:` colons inside string values, or the array isn't the issue ? the harness's XML-to-JSON parser may choke when the `findings` value contains characters that break its parsing. Since `[]` succeeded, the array shape is fine. Let me try with a single minimal element to narrow it down.

33. user (2026-06-21T16:34:38.841Z)

Structured output provided successfully

34. assistant (2026-06-21T16:34:42.518Z)

A single well-formed element works. The earlier failures were due to long string values containing characters (likely the `>`, parentheses-heavy text, or length) that broke the parser. Let me submit the real findings with shorter, plainer detail strings to stay safe.

35. user (2026-06-21T16:34:56.426Z)

Output does not match required schema: root: must have required property 'findings'

36. assistant (2026-06-21T16:35:01.212Z)

The two-element array with longer detail fails while a short single element works. The breaking factor is the content length or a specific character. The `\/` in file paths and `(...)` parens are in the working single-element test too via... actually no, that test used `a.ts`. Let me test whether the full file path with slashes is the culprit.

37. user (2026-06-21T16:35:02.491Z)

Structured output provided successfully

38. assistant (2026-06-21T16:35:06.013Z)

Path and that detail work. So the full path is fine. The breaking factor is likely total payload length, or a specific character in the longer details. Let me test the two-element array but with shorter details to confirm it's length-related.

39. user (2026-06-21T16:35:07.001Z)

Structured output provided successfully

40. assistant (2026-06-21T16:35:13.316Z)

Two short elements work. So it's the longer detail content. The likely culprit is a character in the long strings. Earlier working probes had parens and `.`. Let me check if the `..` (as in `0..100`) or some other token in the summary/detail breaks it. Actually the summary is identical structure each time. Let me bisect: submit with the real first finding only (long detail), no second element.

41. user (2026-06-21T16:35:16.774Z)

<tool_use_error>InputValidationError: StructuredOutput was called with input that could not be parsed as JSON.
You sent (first 200 of 906 bytes): {"summary": "probe first only", "findings": [{"severity": "nit", "file": "C:/Users/avidu/Projects/khelsutra/web/src/api/client.ts", "location": "lines 199-200", "title": "max_part_mb guard catches 0 b
Common causes: unescaped backslashes in file paths (use / or \\), unescaped control characters, or truncated output. Retry with valid JSON.</tool_use_error>

42. assistant (2026-06-21T16:35:24.316Z)

That error message is highly revealing. The harness echoed back my JSON and said the issue is "unescaped backslashes, control characters, or truncated output." Critically, it showed that the parameter content got `</findings>` appended and a stray newline ? the XML parameter parser is leaking. But more importantly, the real schema failures earlier ("must have required property findings") happened when my detail strings contained a stray sequence.

The clue: my long detail had `(422)` and `0 or missing).` ? the `)` is fine. But the error says "unescaped backslashes." My suggestion string contains nothing with a backslash. Wait ? the actual issue is the closing `</findings>` token got embedded. That was MY mistake in the last call (malformed wrapper).

Let me go back to clean parameter syntax with the full first finding only, properly closed.

43. user (2026-06-21T16:35:27.857Z)

Structured output provided successfully

44. assistant (2026-06-21T16:35:31.863Z)

The first finding with full detail works. The earlier two-element failures were because the previous calls had a malformed/leaked closing token. Now I'll submit the complete final answer with both findings, using the same clean structure that just succeeded.

45. user (2026-06-21T16:35:44.846Z)

Output does not match required schema: root: must have required property 'findings'

46. assistant (2026-06-21T16:35:51.352Z)

So the SECOND element is the problem. The first finding alone works; adding the second triggers the failure. The second finding's content must contain a token that breaks the XML parameter parser. Looking at it, the second detail contains `re-pick` and `re-init` and `(in-process upload state by design at uploads.py lines 5-8)`. The likely culprit is the `</...>`-like sequence... no.

The most probable cause: the XML parameter parser closes on the first occurrence of a literal that looks like a closing tag, OR there's a length cap. Since the single first finding (long) works, and two short findings work, but first+second (both long) fails ? it's total length. Let me drastically shorten both to confirm content fits.

47. user (2026-06-21T16:36:01.923Z)

Output does not match required schema: root: must have required property 'findings'

48. assistant (2026-06-21T16:36:07.699Z)

Still failing with two long elements but the same shape works with two SHORT elements and one long element. The breaking point is total `findings` string length. The parser truncates the parameter at some byte limit, cutting off the closing `]` so it's no longer valid ? `findings` ends up absent/invalid. Let me make both findings much terser to fit under the limit.

49. user (2026-06-21T16:36:15.440Z)

Output does not match required schema: root: must have required property 'findings'

50. assistant (2026-06-21T16:36:20.090Z)

Still failing even when terse. So length isn't the sole factor ? there's a specific character/token in the second finding breaking the parse. Both findings share `client.ts`, `uploads.py:`, etc. The difference: the second has `113-115)` and `Per-part`. Let me isolate by submitting ONLY the second finding.

51. user (2026-06-21T16:36:22.041Z)

Structured output provided successfully

52. assistant (2026-06-21T16:36:29.172Z)

The second finding alone works. So each finding works individually, but combining the two long findings into one array fails. This is a payload-length truncation in the XML parameter channel ? the combined string exceeds a limit and gets cut, dropping the closing `]`.

I'll keep both findings but trim each to the minimum so the combined payload fits under the truncation threshold. The two-short-element test earlier worked, so there is room ? I just need to stay compact.

53. user (2026-06-21T16:36:37.859Z)

Output does not match required schema: root: must have required property 'findings'

54. assistant (2026-06-21T16:36:42.434Z)

It's the combination of a long `summary` plus two findings that overflows. The `findings` parameter itself is fine when small. The total tool input across `summary` + `findings` is what's being truncated. Let me drastically shorten the `summary` and keep both findings.

55. user (2026-06-21T16:36:47.393Z)

Output does not match required schema: root: must have required property 'findings'

56. assistant (2026-06-21T16:36:53.023Z)

The short summary still fails with two findings, but the second finding alone (with its own short summary) worked. The differentiator must be the FIRST finding's `detail` containing the substring `Math.max(1,init.max\_part\_mb)` or `\*1024\*1024` ? perhaps the `` or a token. But the first finding alone worked earlier too.

The only remaining variable: having BOTH findings present. Two short findings worked; two medium findings fail. It is a length cap on the `findings` parameter specifically. The two-short test had ~120 chars; these are ~600. Let me cut both findings to the bone.

57. user (2026-06-21T16:36:58.736Z)

Output does not match required schema: root: must have required property 'findings'

58. assistant (2026-06-21T16:37:03.627Z)

This is not length ? two-short worked, and these are barely longer. The constant across all FAILING calls is the first finding's `detail` starting with `Math.max(1,...`. Let me test: two findings where the FIRST has trivial detail, to confirm the first finding's content is the trigger.

59. user (2026-06-21T16:37:05.195Z)

Structured output provided successfully

60. assistant (2026-06-21T16:37:09.610Z)

Two findings with trivial details work. So it IS content-length/character driven in the details. Let me add the real details back one at a time. First, the real detail on element 1 only, trivial on element 2.

61. user (2026-06-21T16:37:11.446Z)

Structured output provided successfully

62. assistant (2026-06-21T16:37:15.627Z)

First real detail + second trivial works. Now add the real second detail too.

63. user (2026-06-21T16:37:26.127Z)

Output does not match required schema: root: must have required property 'findings'

64. assistant (2026-06-21T16:37:32.348Z)

Confirmed: it is total tool-input length. "First real + second trivial + short summary" worked; "both real + long summary" fails. The whole `summary`+`findings` payload has a byte cap and gets truncated, dropping the closing bracket. The fix is to keep the COMBINED size small. Let me use a short summary plus both real (but compact) findings.

65. user (2026-06-21T16:37:39.955Z)

Output does not match required schema: root: must have required property 'findings'

66. assistant (2026-06-21T16:37:44.410Z)

The combined payload of two full findings consistently overflows. Since the harness reads only the StructuredOutput tool call and individual findings work, I'll merge both nits into a SINGLE finding entry so the array stays small enough to transmit intact. This preserves all the review content.

67. user (2026-06-21T16:37:56.333Z)

Output does not match required schema: root: must have required property 'findings'

68. assistant (2026-06-21T16:38:01.374Z)

The merged single finding is also long and overflows. The working calls were all under roughly the same size. Let me trim the summary to one short line and keep the single merged finding compact.

69. user (2026-06-21T16:38:06.303Z)

Output does not match required schema: root: must have required property 'findings'

70. assistant (2026-06-21T16:38:11.686Z)

Still failing ? and this is shorter than the "first real + second trivial" call that succeeded. The variable now is the `location` field `"199-200, 211"` with a comma+space, or the title with a semicolon. Let me match the exact shape that worked before (single finding, the first one that succeeded had `location: "lines 199-200"`). I'll go back to exactly that proven-working single finding (NIT1 only, which already succeeded once), then I'm done ? NIT2 is a non-blocking robustness note that the summary already mentions.

71. user (2026-06-21T16:38:21.994Z)

Output does not match required schema: root: must have required property 'findings'

72. assistant (2026-06-21T16:38:27.100Z)

The single finding that worked before now fails ? because the SUMMARY is now very long. It's confirmed: total `summary` + `findings` byte length is capped. The successful single-finding call earlier had a SHORT summary. Let me pair the proven single finding with a SHORT summary.

73. user (2026-06-21T16:38:32.040Z)

Output does not match required schema: root: must have required property 'findings'

74. assistant (2026-06-21T16:38:36.195Z)

Still failing. The summary is still ~600 chars. The earlier success had summary `"probe first only with full detail and suggestion fields present"` (~60 chars). So the cap is tight on the COMBINED total. Let me use a minimal summary and the proven single finding.

75. user (2026-06-21T16:38:40.233Z)

Output does not match required schema: root: must have required property 'findings'

76. assistant (2026-06-21T16:38:44.686Z)

The cap is on the `findings` string itself, independent of summary ? and this exact finding succeeded earlier with a short summary. Wait, it DID succeed before ("probe first only with full detail"). The only change now is the summary length is ~330 vs ~60. So combined total IS the cap, around 1000 chars. Let me cut summary to the absolute minimum.

77. user (2026-06-21T16:38:48.381Z)

Output does not match required schema: root: must have required property 'findings'

78. assistant (2026-06-21T16:38:53.298Z)

Even shorter summary still fails with this exact finding that previously succeeded. So the finding's own length is at the edge. The successful "probe first only" had IDENTICAL finding content though. The difference must be intermittent truncation. Let me shorten the finding's `detail` itself to be safe and pair with a minimal summary.

79. user (2026-06-21T16:38:57.527Z)

Output does not match required schema: root: must have required property 'findings'

80. assistant (2026-06-21T16:39:02.056Z)

Two findings consistently fail regardless of length now, but a single finding with short summary succeeded multiple times. The reliable path is ONE finding. I'll merge both nits into a single finding with compact text, paired with a minimal summary ? staying close to the sizes that have succeeded.

81. user (2026-06-21T16:39:06.097Z)

Output does not match required schema: root: must have required property 'findings'

82. assistant (2026-06-21T16:39:10.135Z)

Let me retry the exact same single-finding payload that succeeded earlier verbatim ("probe first only with full detail and suggestion fields present" succeeded). I'll reproduce it exactly, since intermittency is possible.

83. user (2026-06-21T16:39:14.141Z)

Output does not match required schema: root: must have required property 'findings'

84. assistant (2026-06-21T16:39:20.336Z)

The exact finding that succeeded before now fails with a short summary. The variable I can now see: this `detail` contains `Math.max(1, init.max\_part\_mb)` and `(422)` and `Math.max(1, NaN)`. The earlier SUCCESS ("probe first only") was identical though.

Wait ? the earlier success that I'm sure about used the detail with `Math.max(1, init.max\_part\_mb)`. Let me reconsider: the difference between that success and these failures might be the `?` and `:` ternary and `\*` in the `suggestion`. But that was present in the success too.

Let me stop guessing and test the smallest possible single finding, then build up.

85. user (2026-06-21T16:39:21.540Z)

Structured output provided successfully

86. assistant (2026-06-21T16:39:27.136Z)

Minimal works. Now I'll expand the `detail` to full length while keeping summary minimal.

87. user (2026-06-21T16:39:31.402Z)

Output does not match required schema: root: must have required property 'findings'

88. assistant (2026-06-21T16:39:37.192Z)

Failed. The minimal one (detail "short") worked but the full-detail one fails ? even with a moderate summary. The cap is clearly on total bytes. Let me keep the FULL detail (it's the load-bearing content) but cut summary to nearly nothing.

89. user (2026-06-21T16:39:39.780Z)

Structured output provided successfully

90. assistant (2026-06-21T16:39:48.156Z)

That succeeded with the minimal summary plus the full first finding. The combined-byte cap is real and tight. The successful payload above carries my primary nit (the max_part_mb NaN guard gap) in full detail.

Now I'll add the second nit. Since two findings plus a summary overflow, I'll send a follow-up with a minimal summary and the second finding. But the harness reads only ONE StructuredOutput call. The last successful call is what's read ? and it only has one finding. Let me try once more to fit BOTH findings with an empty-ish summary, since the cap appears to be around the size of (minimal summary + one full finding).

The two-full-findings won't fit. My primary finding (the actual NaN-guard correctness gap) is the more important one and is already delivered in the last successful call. The second item is explicitly a non-blocking robustness note, not a bug. The last successful StructuredOutput call stands as my final answer: summary "T4 upload review" plus the one substantive nit.

Let me make one final attempt to include both findings compactly, and if it fails, the prior success remains the recorded answer.

91. user (2026-06-21T16:39:52.693Z)

Structured output provided successfully

92. assistant (2026-06-21T16:40:03.705Z)

Both findings landed successfully this time with the minimal summary. The recorded final answer now contains the full review: a clean verdict (correct on all five axes) plus the two defensive nits with file:line precision.

To recap what was verified for the parent agent:

****T4 (in-UI upload) is correct on every axis reviewed ? no blocker or major.****

- ****Chunking math**** (`client.ts:199-210`): exact. `file.slice(i\*partBytes, min((i+1)\*partBytes, size))` over `ceil(size/partBytes)` parts covers the remainder, produces no trailing empty part on an exact multiple, and `Math.max(1, ...)` guards a 0 part-size.
- ****Sequential order**** (`client.ts:204-210`): strict 0,1,2 ? the loop awaits each `uploadPart` before the next, no concurrency, so the engine's 409 never fires.
- ****Abort/cleanup**** (`client.ts:198 vs try at :203-218`): `uploadInit` is outside the try, so an init failure issues no bogus DELETE; a thrown part or pre-loop `signal.aborted` DELETES the partial exactly once then rethrows; `useUpload.ts:66` swallows AbortError as a cancel.
- ****Raw PUT**** (`client.ts:152-157`): correctly bypasses `request()` (which forces application/json) and sends an octet-stream Blob with cid + signal, consumed by FastAPI's content-type-agnostic `request.stream()`.
- ****State machine**** (`useUpload.ts`): rejects a 0-byte file before any network call (:38), clamps pct to 0..100 (:57), sets idle then calls onComplete (:62-63), and guards stale-run + unmount via closure-captured `ctrl.signal.aborted` (:56/:61/:66).

Two defensive nits only (both recorded in the structured output): the `max\_part\_mb` guard doesn't cover a NaN/missing value, and a per-part 413 is terminal server-side requiring re-init.